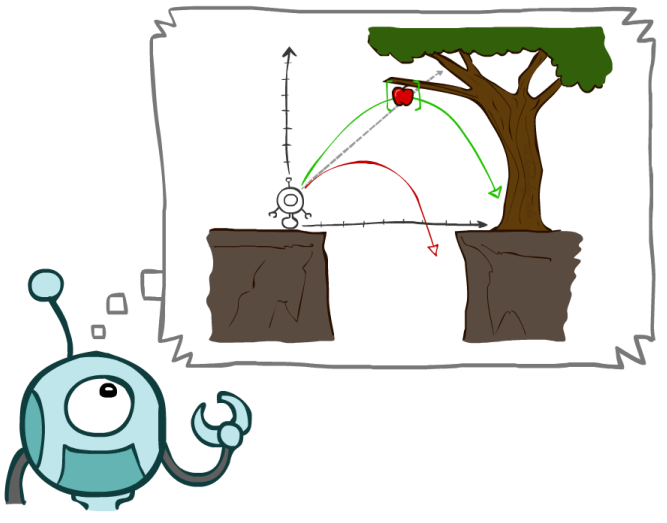
SI3003 - Inteligencia Artificial

CLASE 2 – RESOLUCIÓN DE PROBLEMAS MEDIANTE BÚSQUEDA

Hoy

- Agentes de planificación
- Problemas de búsqueda
- Métodos de búsqueda no informada
  - Búsqueda en profundidad (DFS)
  - Búsqueda en amplitud (BFS)
  - Búsqueda de costo uniforme (UCS)
- Métodos de búsqueda informada
  - A*
  - Heurísticas



Agentes de planificación

Recordemos la Clase 1: un agente **basado en objetivos** genera secuencias de acciones, predice sus estados resultantes, evalúa el objetivo en cada uno y elige la primera acción de la secuencia ganadora. Esos 4 pasos son, informalmente, un problema de búsqueda. Hoy formalizamos cómo ejecutarlos sin enumerar cada secuencia posible.

Agentes de reflejo

Los agentes de reflejo

- eligen acciones con base en el percept actual;
- pueden tener un modelo del estado actual del mundo;
- no consideran las consecuencias futuras de sus acciones;
- consideran solo **cómo es el mundo ahora**.

Los reflejos alcanzan para tomar acciones racionales en entornos *totalmente observables*, *determinísticos* y *conocidos*.

Sin embargo, son **difíciles de implementar** para tareas complejas, ya que determinar la acción inmediata que produce el mejor resultado a largo plazo se expresa más naturalmente en términos de planificación.

Agentes de resolución de problemas

Supuestos:

- Entorno de *un solo agente*, *observable*, *determinístico* y *conocido*.

Los agentes de resolución de problemas

- toman decisiones con base en las consecuencias (hipotéticas) de sus acciones, considerando **cómo podría ser el mundo**;
- deben tener un modelo de cómo evoluciona el mundo en respuesta a las acciones;
- formulan un objetivo, de forma explícita.

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
              state, some description of the current world state
              goal, a goal, initially null
              problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
      goal ← FORMULATE-GOAL(state)
      problem ← FORMULATE-PROBLEM(state, goal)
      seq ← SEARCH(problem)
      if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

Figure 3.1 A simple problem-solving agent. It first formulates a goal and a problem, searches for a sequence of actions that would solve the problem, and then executes the actions one at a time. When this is complete, it formulates another goal and starts over.

Resolución offline vs. online

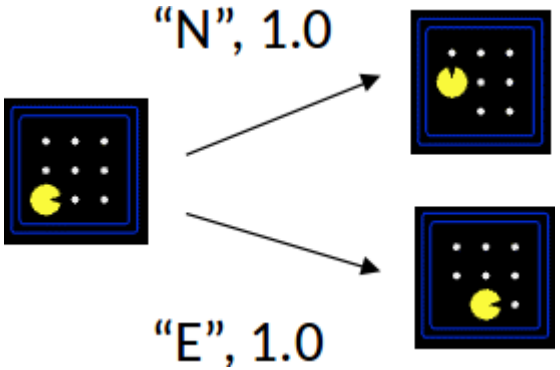
- Los agentes de resolución de problemas son **offline**. La solución se ejecuta "con los ojos cerrados", ignorando los percepts.
- La resolución **online** implica actuar sin conocimiento completo. En este caso, la secuencia de acciones puede recalcularse en cada paso.

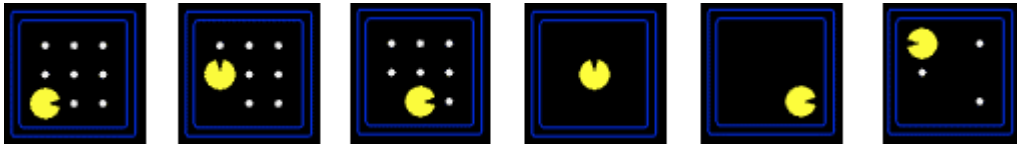
Problemas de búsqueda

Problemas de búsqueda

Un **problema de búsqueda** consiste de los siguientes componentes:

- Una representación de los *estados* del agente y su entorno.
- El *estado inicial* del agente.
- Una descripción de las *acciones* disponibles para el agente dado un estado s, denotada $\text{acciones}(s)$.
- Un *modelo de transición* que devuelve el estado $s' = \text{resultado}(s, a)$ que resulta de ejecutar la acción a en el estado s.
    - Decimos que s' es un .italic[sucesor] de s si existe una acción aceptable de s a s'.





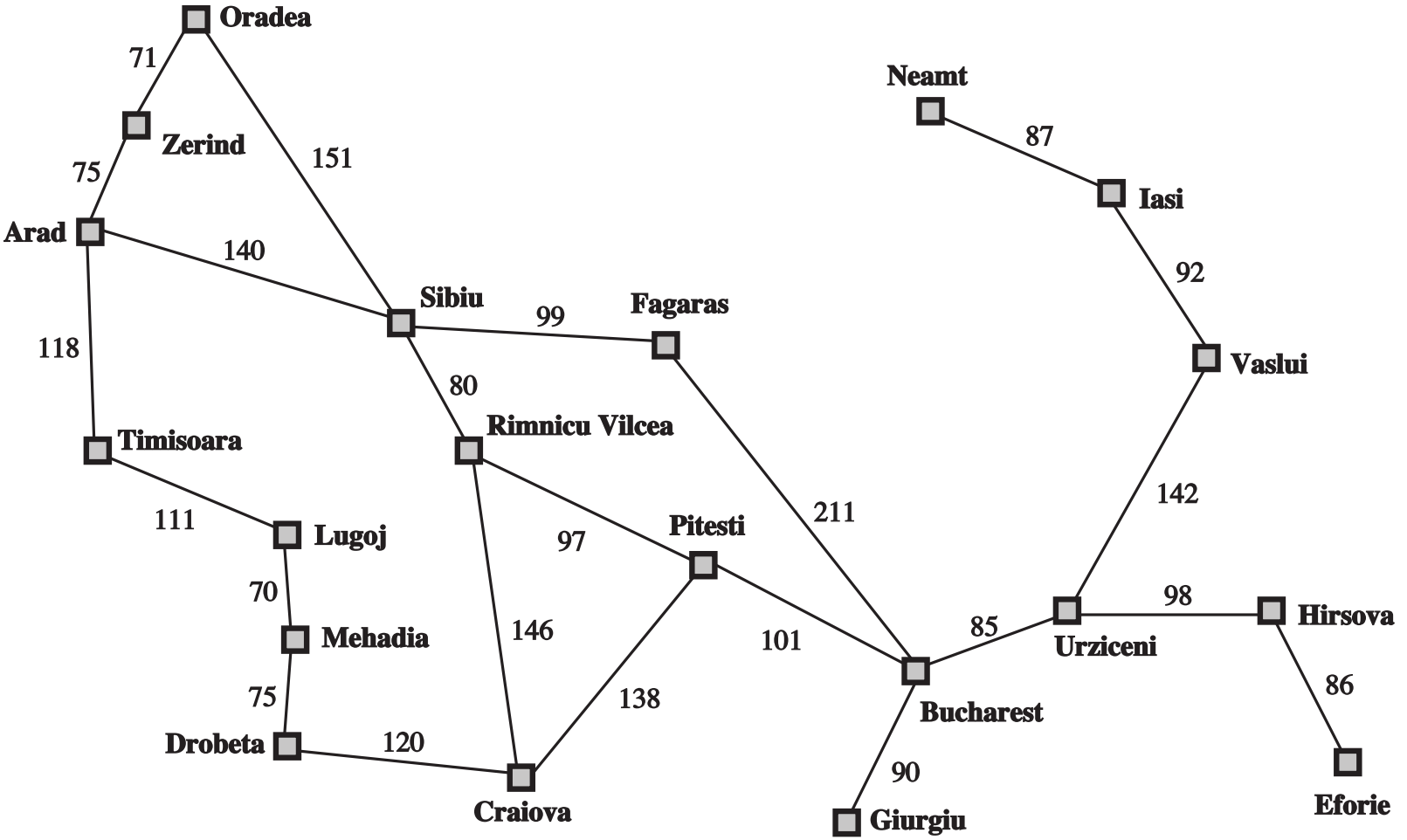
- Juntos, el estado inicial, las acciones y el modelo de transición definen el **espacio de estados** del problema, es decir, el conjunto de todos los estados alcanzables desde el estado inicial por cualquier secuencia de acciones.
  - El espacio de estados forma un grafo dirigido:
    - nodos = estados
    - enlaces = acciones
  - Un camino es una secuencia de estados conectados por acciones.
- Una *prueba de objetivo* que determina si la solución del problema se alcanza en el estado s.
- Un *costo del camino* que asigna un valor numérico a cada camino.
  - En este curso, además asumiremos que el costo del camino corresponde a la suma de .italic[costos de paso] positivos $c(s, a, s')$ asociados a la acción a en s que lleva a s'.

Una *solución* a un problema es una secuencia de acciones que lleva del estado inicial a un estado objetivo.

- La calidad de una solución se mide con la función de costo del camino.
- Una **solución óptima** tiene el menor costo del camino entre todas las soluciones.

¿Qué pasa si el entorno es parcialmente observable? ¿Si es no determinístico?

Ejemplo: viajando por Rumania



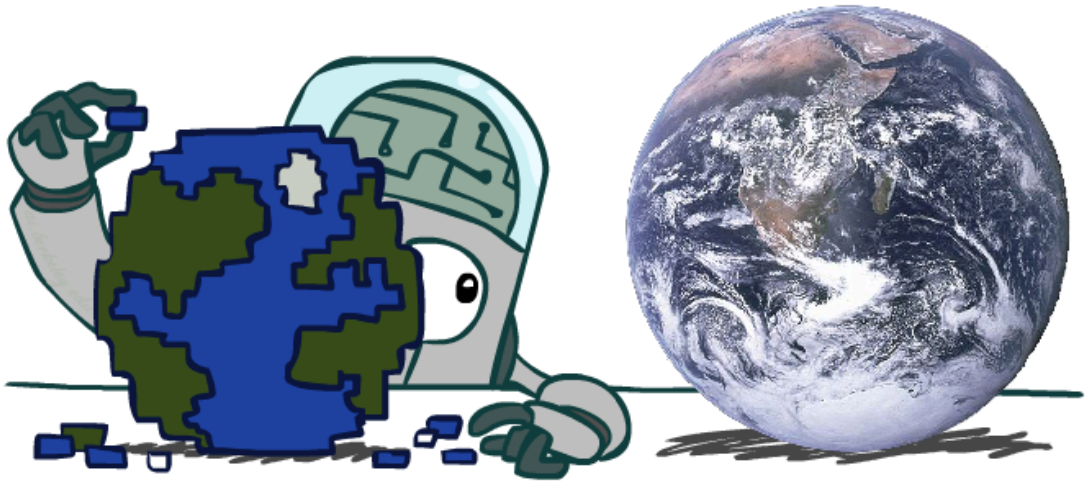
¿Cómo ir de Arad a Bucarest?

- Representación de estados: la ciudad en la que estamos.
  - $s \in \{\text{en}(\text{Arad}), \text{en}(\text{Bucarest}), \ldots\}$
- Estado inicial = la ciudad donde empezamos.
  - $s_0 = \text{en}(\text{Arad})$
- Acciones = ir de la ciudad actual a las ciudades directamente conectadas con ella.
  - $\text{acciones}(s_0) = \{\text{ir a}(\text{Sibiu}), \text{ir a}(\text{Timisoara}), \text{ir a}(\text{Zerind})\}$
- Modelo de transición = la ciudad a la que llegamos tras viajar a ella.
  - $\text{resultado}(\text{en}(\text{Arad}), \text{ir a}(\text{Zerind})) = \text{en}(\text{Zerind})$
- Prueba de objetivo: si estamos en Bucarest.
  - $s \in \{\text{en}(\text{Bucarest})\}$
- Costo de paso: distancias entre ciudades.

Eligiendo un espacio de estados

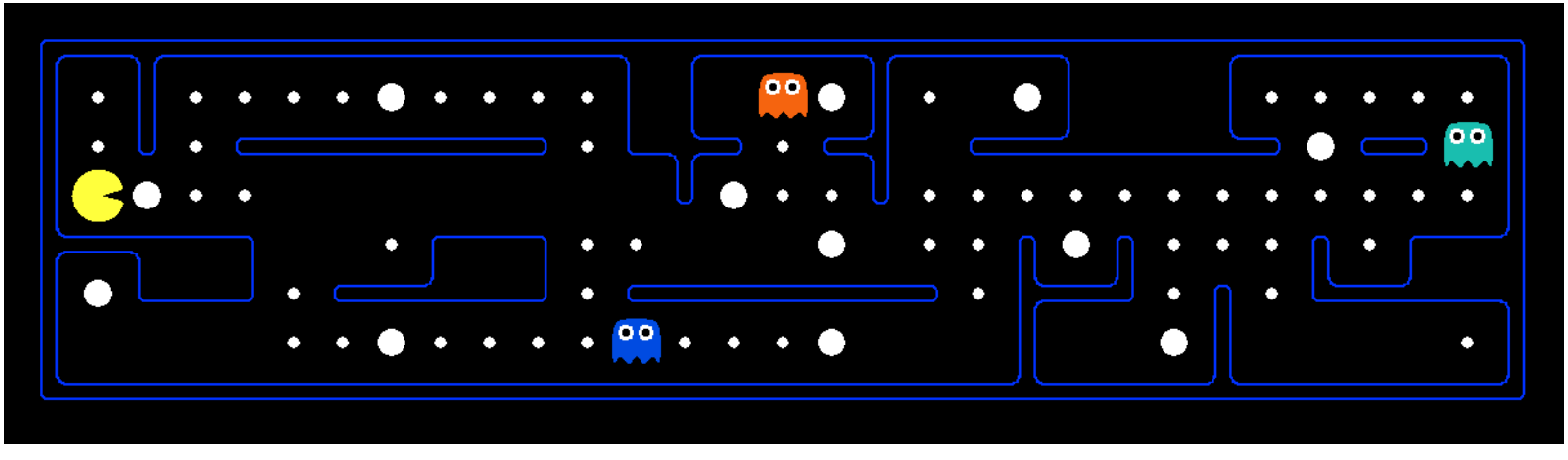
El mundo real es absurdamente **complejo**.

- El *estado del mundo* incluye cada detalle del entorno.
- Un *estado de búsqueda* conserva solo los detalles necesarios para planificar.



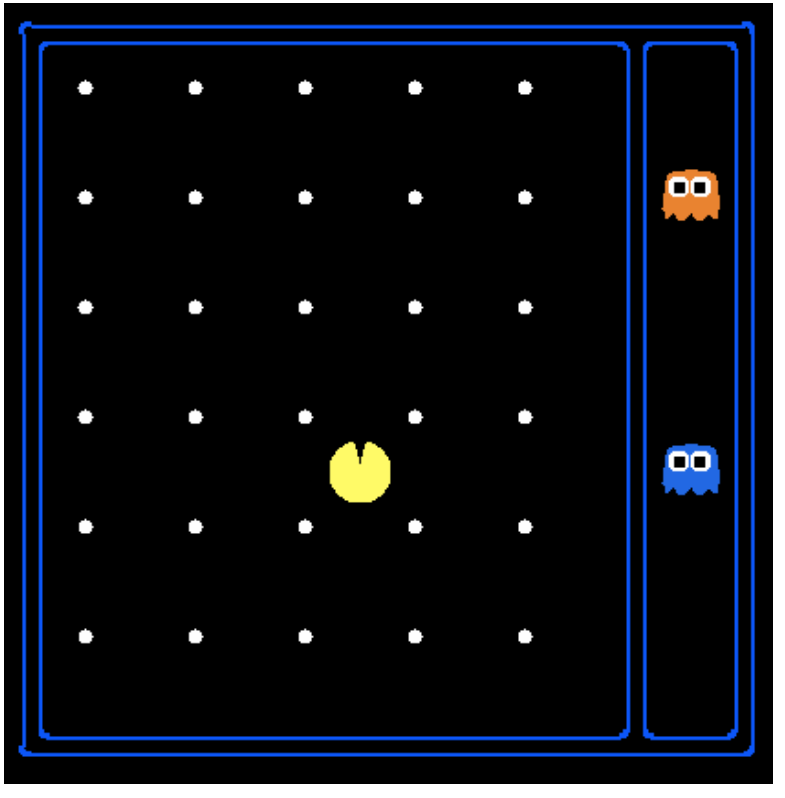
Ejemplo: comerse todos los puntos

- Estados: $\{(x, y), \text{booleanos de puntos}\}$
- Acciones: NSEO
- Transición: actualizar la ubicación y, posiblemente, un booleano de punto
- Prueba de objetivo: todos los puntos en falso



Tamaño del espacio de estados

- *Estado del mundo*:
  - Posiciones del agente: 120
  - Cantidad de puntos: 30
  - Posiciones de fantasmas: 12
  - Orientación del agente: NSEO
- *¿Cuántos hay?*
  - ¿Estados del mundo?
    - $120 \times 2^{30} \times 12^2 \times 4$
  - ¿Estados para comerse-todos-los-puntos?
    - 120×2^{30}

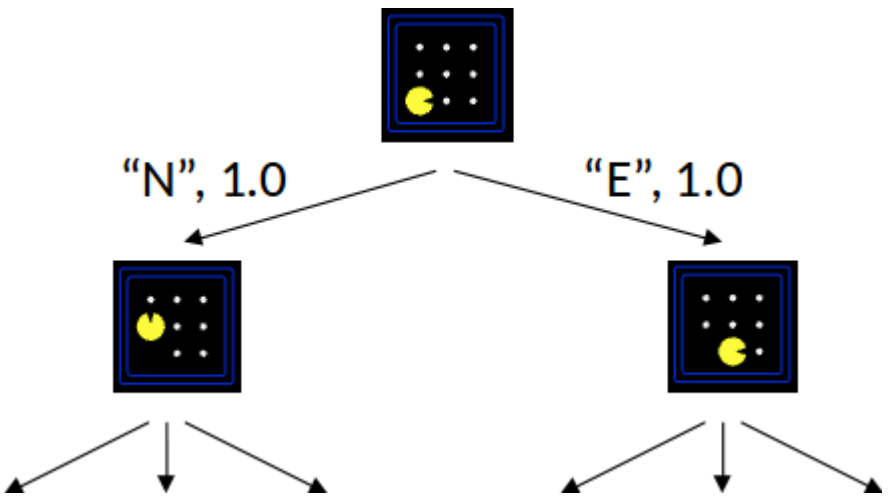


Árboles de búsqueda

El conjunto de secuencias aceptables que parten del estado inicial forma un **árbol de búsqueda**.

- Los nodos corresponden a estados en el espacio de estados; el estado inicial es la raíz.
- Las ramas corresponden a acciones aplicables, y los nodos hijos corresponden a los sucesores.

Para la mayoría de los problemas, nunca podemos construir el árbol completo. ¡Aun así queremos encontrar alguna rama óptima!



Algoritmos de búsqueda en árbol

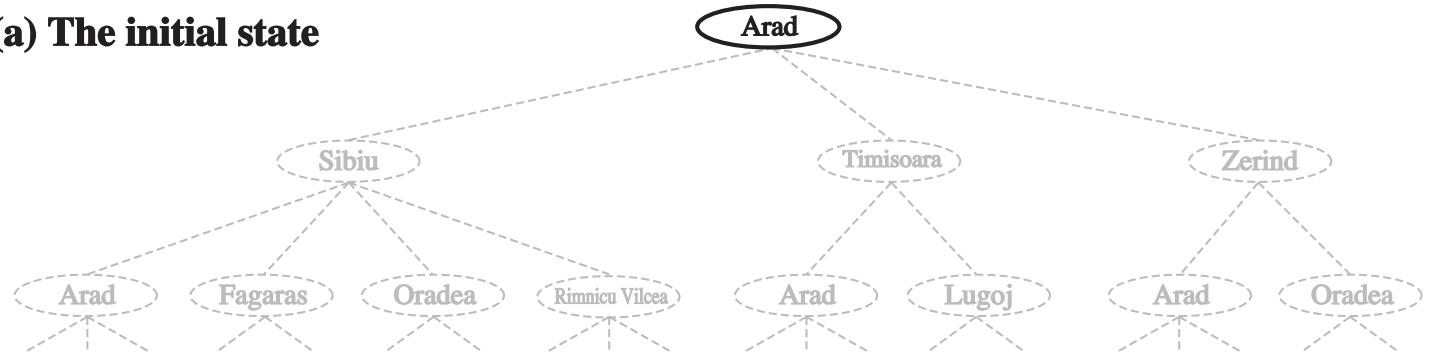
```
function TREE-SEARCH( problem, fringe) returns a solution, or failure
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST(problem, STATE(node)) then return node
        fringe ← INSERTALL(EXPAND(node, problem), fringe)
```

Ideas importantes

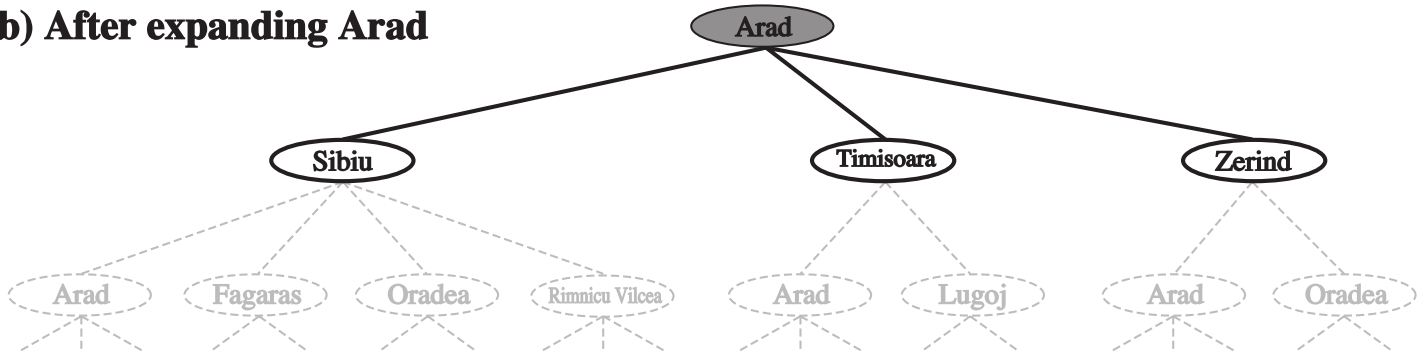
- Frontera (o *fringe*) de planes parciales bajo consideración
- Expansión
- Exploración

¿Qué nodos de la frontera explorar? ¿Cómo expandir la menor cantidad de nodos posible, sin dejar de alcanzar el objetivo?

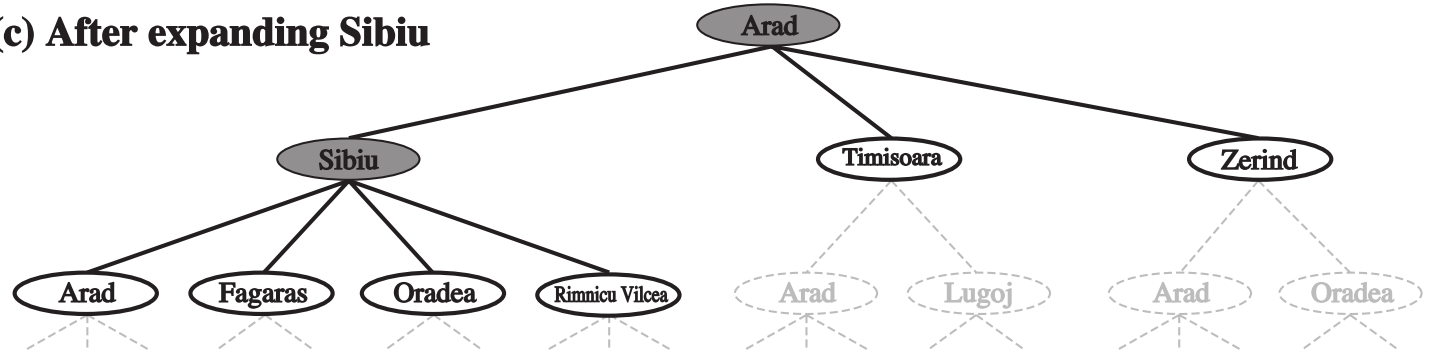
(a) The initial state



(b) After expanding Arad



(c) After expanding Sibiu



Estrategias de búsqueda no informada

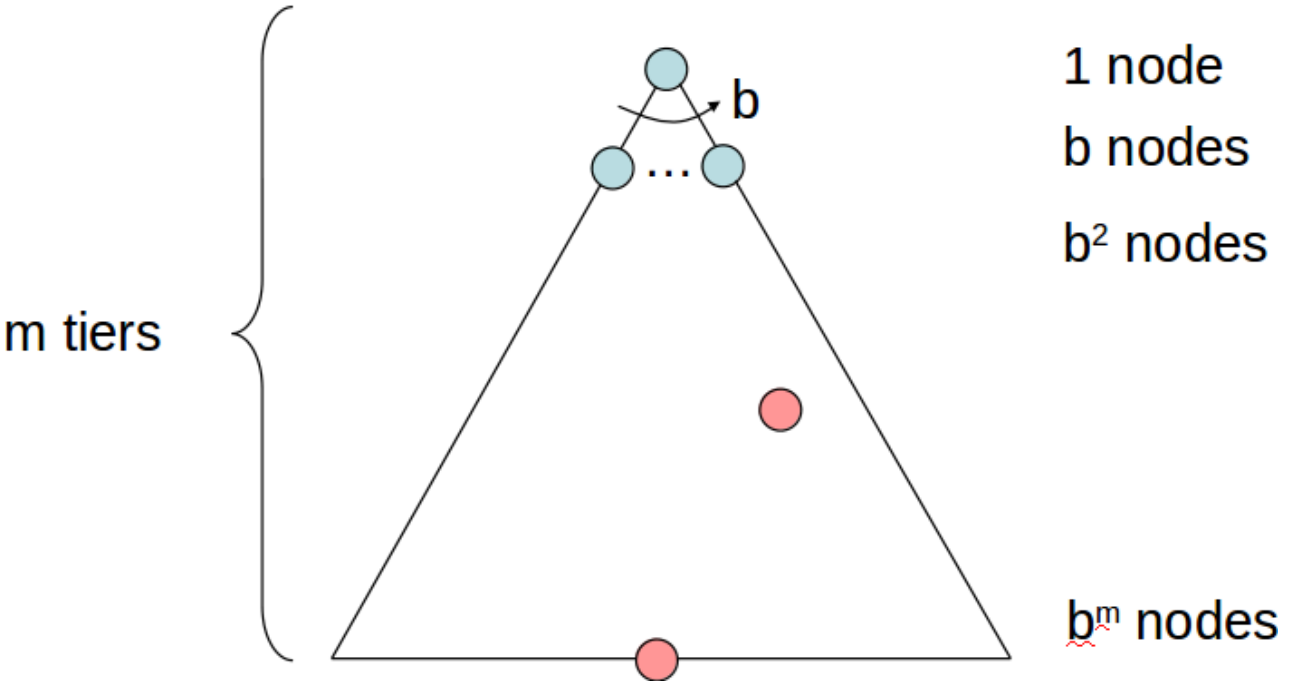
Las estrategias de búsqueda **no informada** usan solo la información disponible en la definición del problema. No saben si un estado se ve más prometedor que otro.

Estrategias

- Búsqueda en profundidad (DFS)
- Búsqueda en amplitud (BFS)
- Búsqueda de costo uniforme (UCS)
- Profundización iterativa

Propiedades de las estrategias de búsqueda

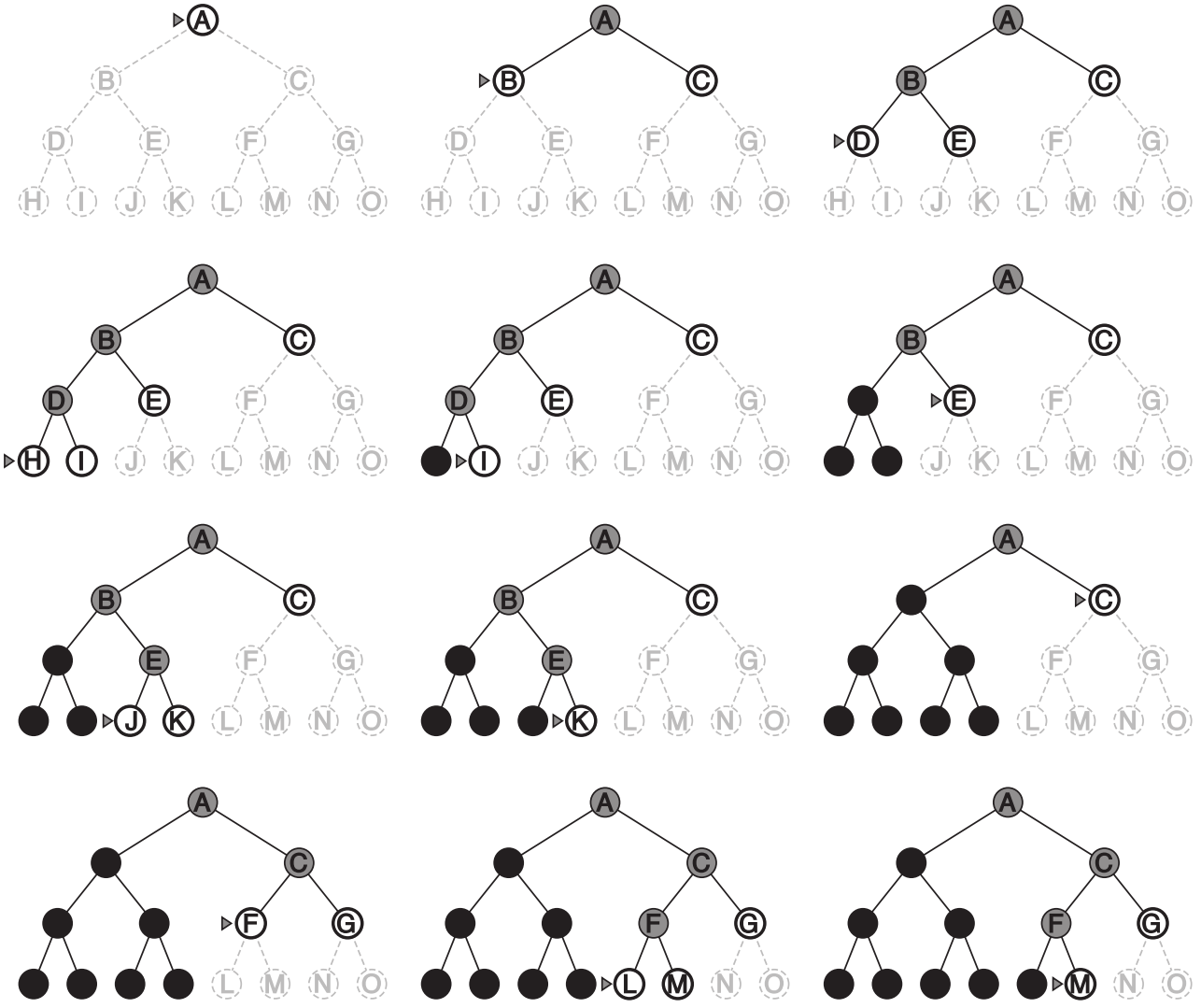
- Una estrategia se define eligiendo el **orden de expansión**.
- Las estrategias se evalúan según las siguientes dimensiones:
  - .italic[Completitud]: ¿siempre encuentra una solución si existe alguna?
  - .italic[Optimalidad]: ¿siempre encuentra la solución de menor costo?
  - .italic[Complejidad temporal]: ¿cuánto tarda en encontrar una solución?
  - .italic[Complejidad espacial]: ¿cuánta memoria necesita para hacer la búsqueda?
- El tiempo y la complejidad se miden en función de
  - b: factor de ramificación máximo del árbol de búsqueda
  - d: profundidad de la solución de menor costo
    - la profundidad de s se define como el número de acciones desde el estado inicial hasta s .
  - m: longitud máxima de cualquier camino en el espacio de estados (puede ser ∞)

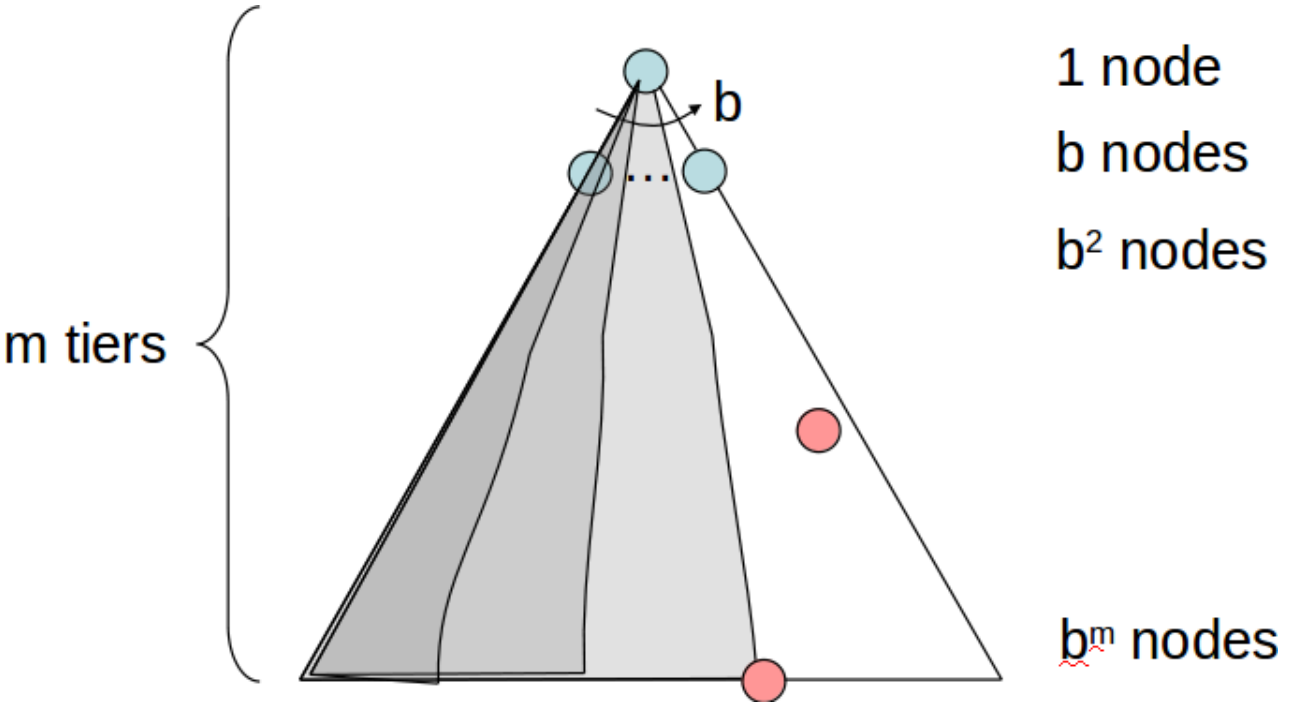

m tiers
b
1 node
b nodes
b² nodes
bᵐ nodes

Búsqueda en profundidad (DFS)



- *Estrategia*: expandir el nodo más profundo de la frontera.
- *Implementación*: la frontera es una **pila LIFO**.

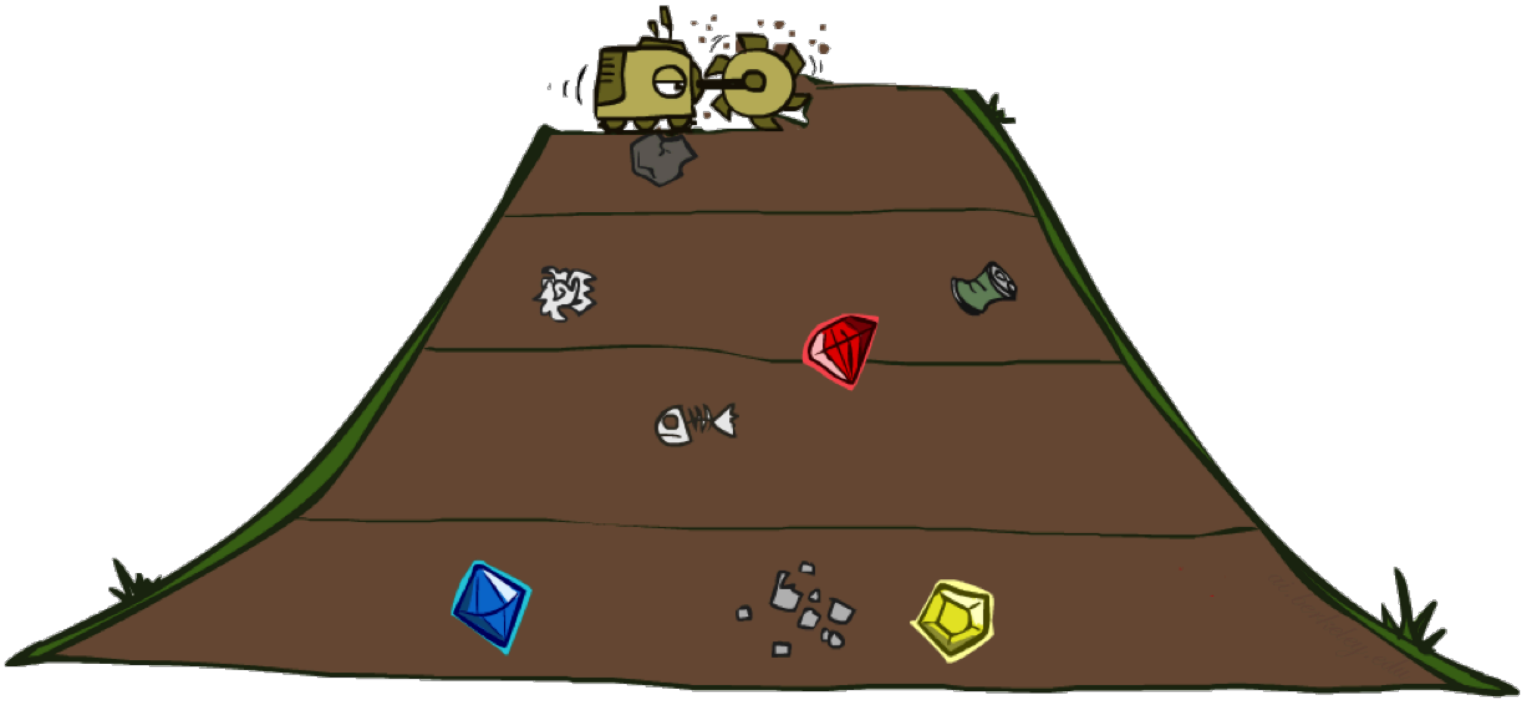



m tiers
b
1 node
b nodes
b² nodes
bᵐ nodes

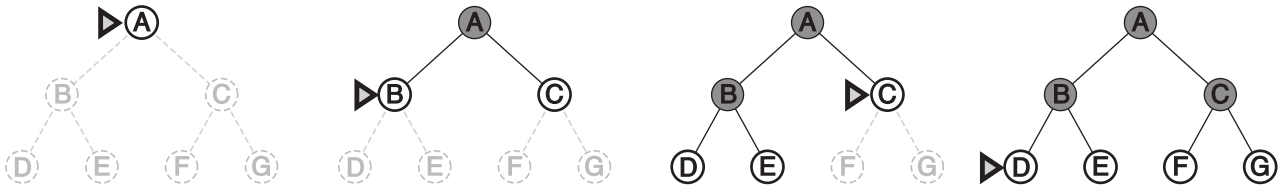
Propiedades de DFS

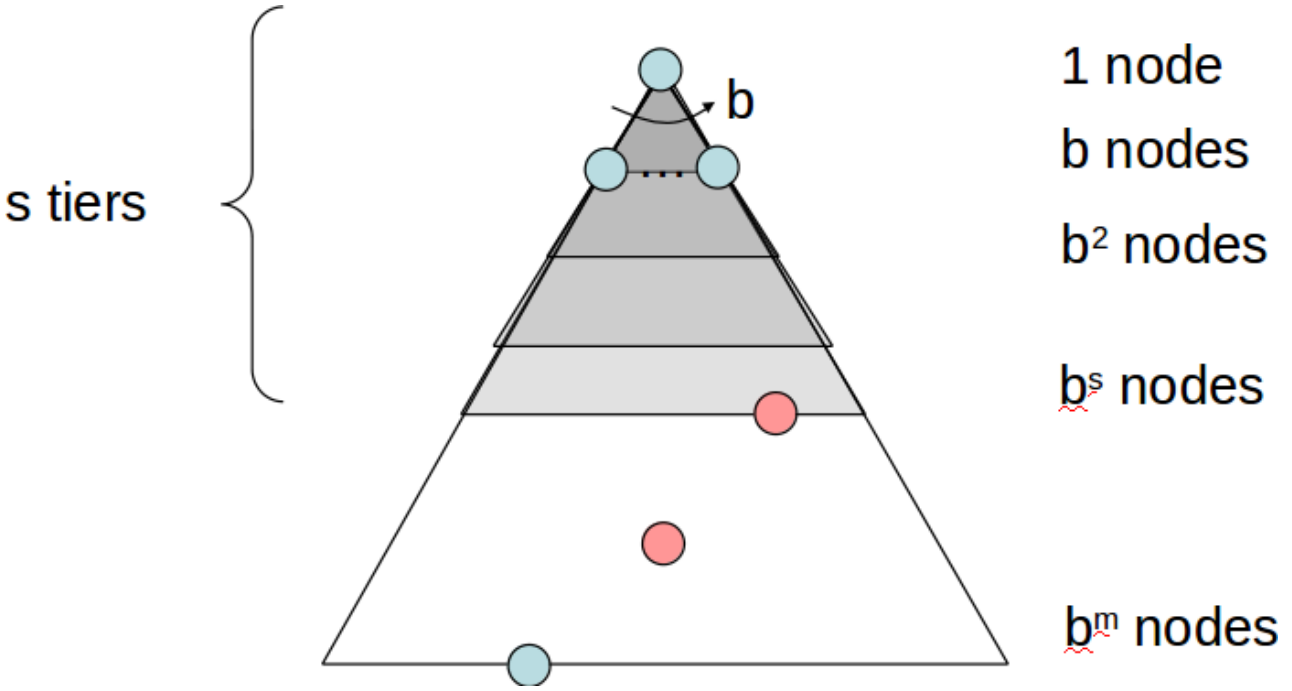
- *Completitud*:
  - m podría ser infinito, así que solo se cumple si evitamos ciclos (más sobre esto adelante).
- *Optimalidad*:
  - No, DFS encuentra la solución más a la izquierda, sin importar su profundidad o costo.
- *Complejidad temporal*:
  - Puede generar el árbol completo (o buena parte de él, independientemente de d). Por lo tanto $O(b^m)$, que puede ser mucho mayor que el tamaño del espacio de estados.
- *Complejidad espacial*:
  - Solo almacena a los hermanos del camino hacia la raíz, por lo tanto $O(bm)$.
  - Cuando todos los descendientes de un nodo ya se visitaron, el nodo se puede eliminar de memoria.

Búsqueda en amplitud (BFS)



- *Estrategia*: expandir el nodo más superficial de la frontera.
- *Implementación*: la frontera es una **cola FIFO**.




s tiers
b
1 node
b nodes
b² nodes
bˢ nodes
bᵐ nodes

Propiedades de BFS

- *Completitud*:
  - Si el nodo objetivo más superficial está a una profundidad finita d, BFS eventualmente lo encontrará tras generar todos los nodos menos profundos (siempre que b sea finito).
- *Optimalidad*:
  - El objetivo más superficial no es necesariamente el óptimo.
  - BFS es óptima solo si el costo del camino es una función no decreciente de la profundidad del nodo.
- *Complejidad temporal*:
  - Si la solución está a profundidad d, el número total de nodos generados antes de encontrarla es $b + b^2 + b^3 + ... + b^d = O(b^d)$
- *Complejidad espacial*:
  - El número de nodos a mantener en memoria es el tamaño de la frontera, que será máximo en el último nivel. Es decir, $O(b^d)$

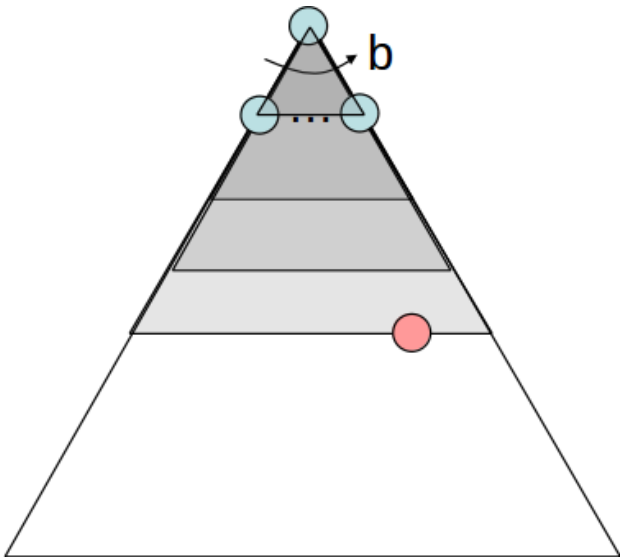
(demo)

Profundización iterativa

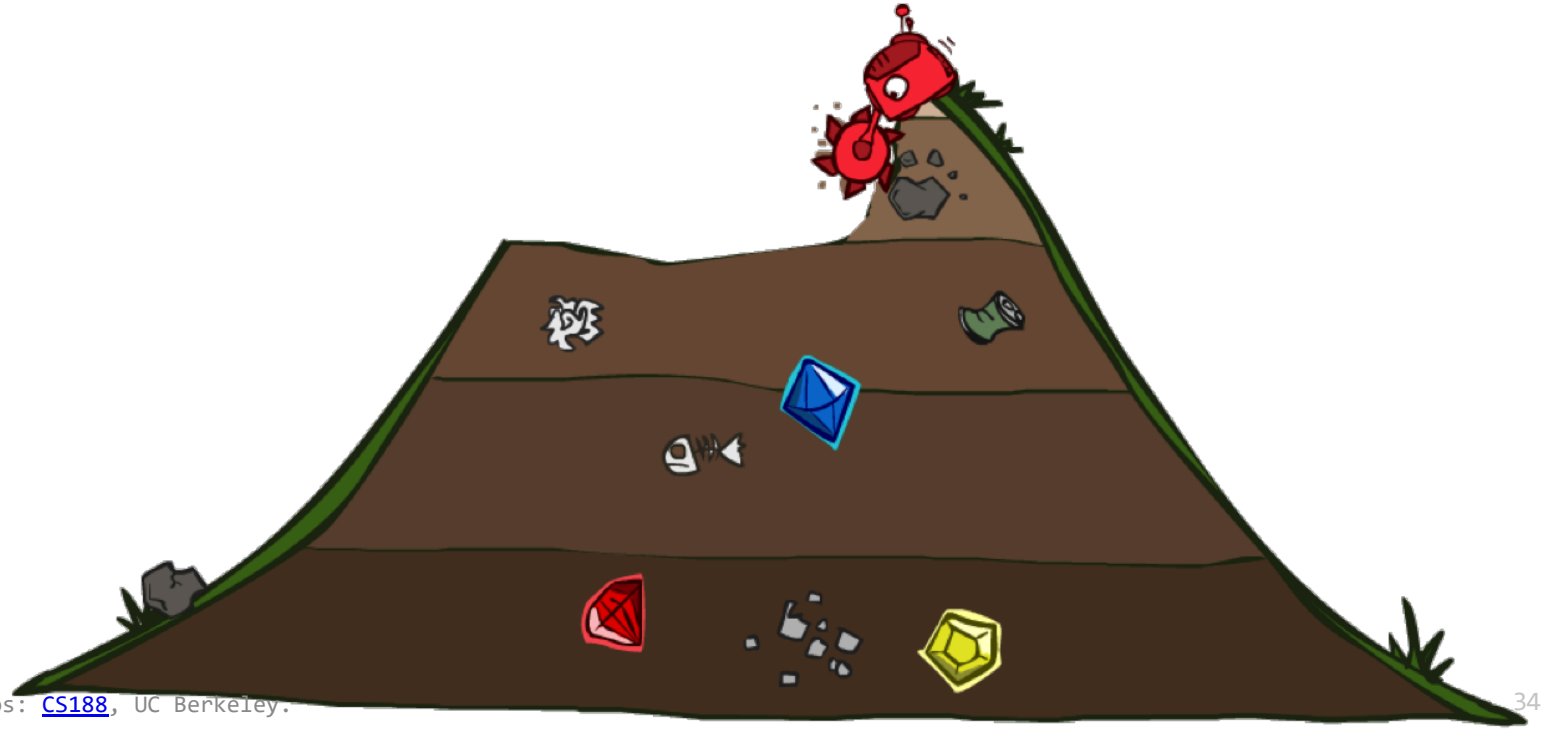
Idea: obtener las ventajas de espacio de DFS con las ventajas de tiempo/solución superficial de BFS.

- Correr DFS con límite de profundidad 1.
- Si no hay solución, correr DFS con límite de profundidad 2.
- Si no hay solución, correr DFS con límite de profundidad 3.
  - ...

- ¿Cuáles son las propiedades de la profundización iterativa?
- ¿No es este proceso innecesariamente redundante?



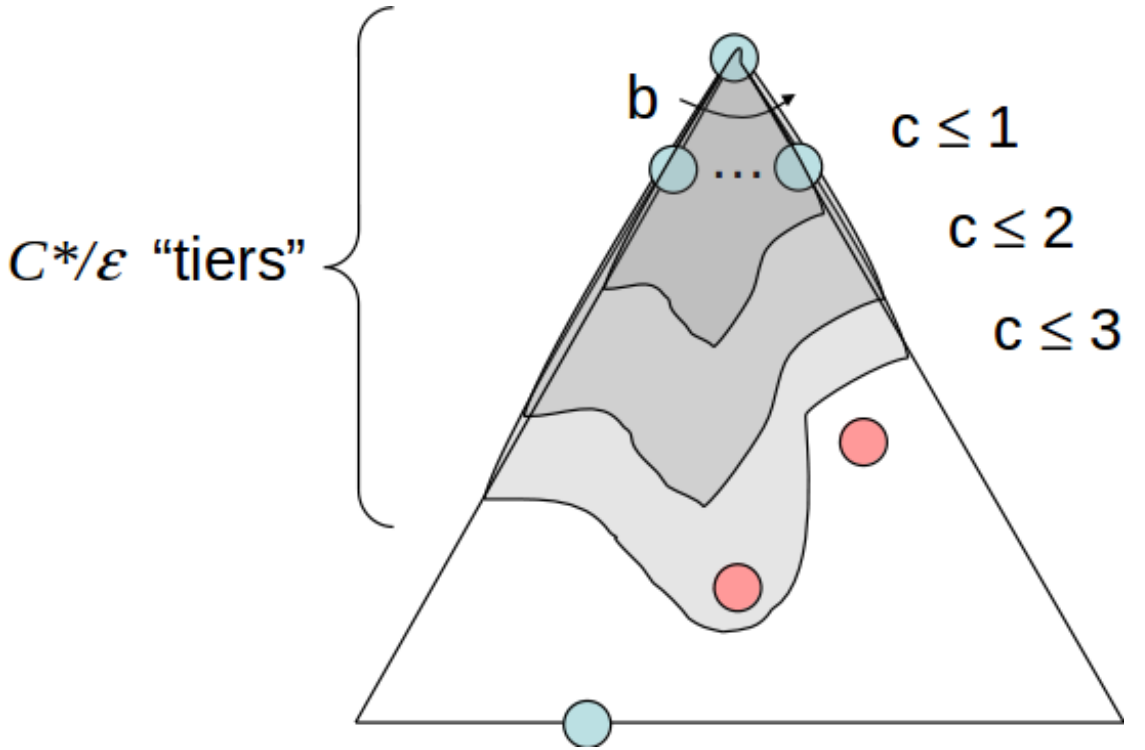
Búsqueda de costo uniforme (UCS)



- *Estrategia*: expandir el nodo más barato de la frontera.
- *Implementación*: la frontera es una

  cola de prioridad

  , usando el costo acumulado $g(n)$ desde el estado inicial hasta el nodo n como prioridad.

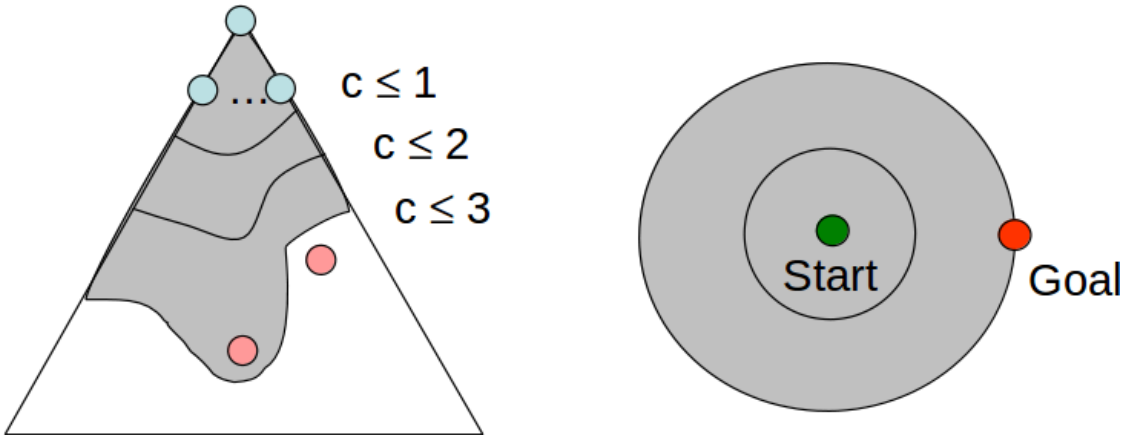

C^*/ε "tiers"
b
...
c ≤ 1
c ≤ 2
c ≤ 3

Propiedades de UCS

- *Completitud*:
  - Sí, si todos los costos de paso cumplen $c(s, a, s') \geq \epsilon > 0$. (¿Por qué?)
- *Optimalidad*:
  - Sí, ya que UCS expande los nodos en orden de su costo de camino óptimo.
- *Complejidad temporal*:
  - Asumamos que C^* es el costo de la solución óptima y que todos los costos de paso son $\geq \epsilon$.
  - La "profundidad efectiva" es entonces, aproximadamente, C^*/ϵ.
  - La complejidad temporal en el peor caso es $O(b^{C^*/\epsilon})$.
- *Complejidad espacial*:
  - El número de nodos a mantener es el tamaño de la frontera, es decir, tantos como en el último nivel: $O(b^{C^*/\epsilon})$.

(demo)

Estrategias de búsqueda informada



Uno de los **problemas de UCS** es que explora el espacio de estados en *todas las direcciones*, sin aprovechar información sobre la ubicación (plausible) del nodo objetivo.

Las estrategias de búsqueda **informada** buscan resolver este problema expandiendo los nodos de la frontera en orden decreciente de *deseabilidad*.

- Búsqueda voraz (*greedy*)
- A*

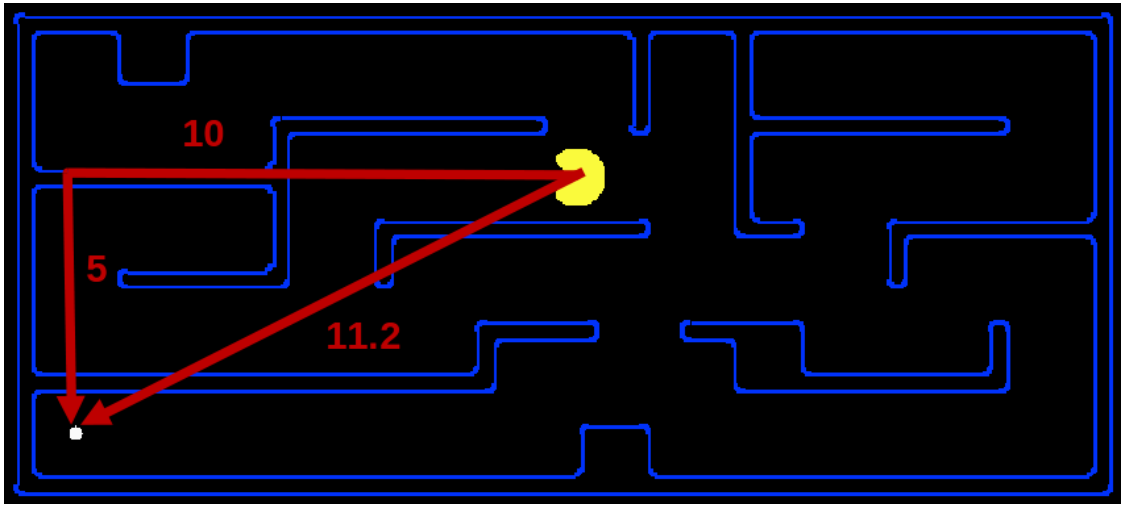
Búsqueda voraz (greedy)



Heurísticas

Una **heurística** (o función de evaluación) $h(n)$ es:

- una función que *estima* el costo del camino más barato desde el nodo n hasta un estado objetivo;
  - $h(n) \geq 0$ para todo nodo n
  - $h(n) = 0$ para un estado objetivo.
- se diseña para un problema de búsqueda *particular*.

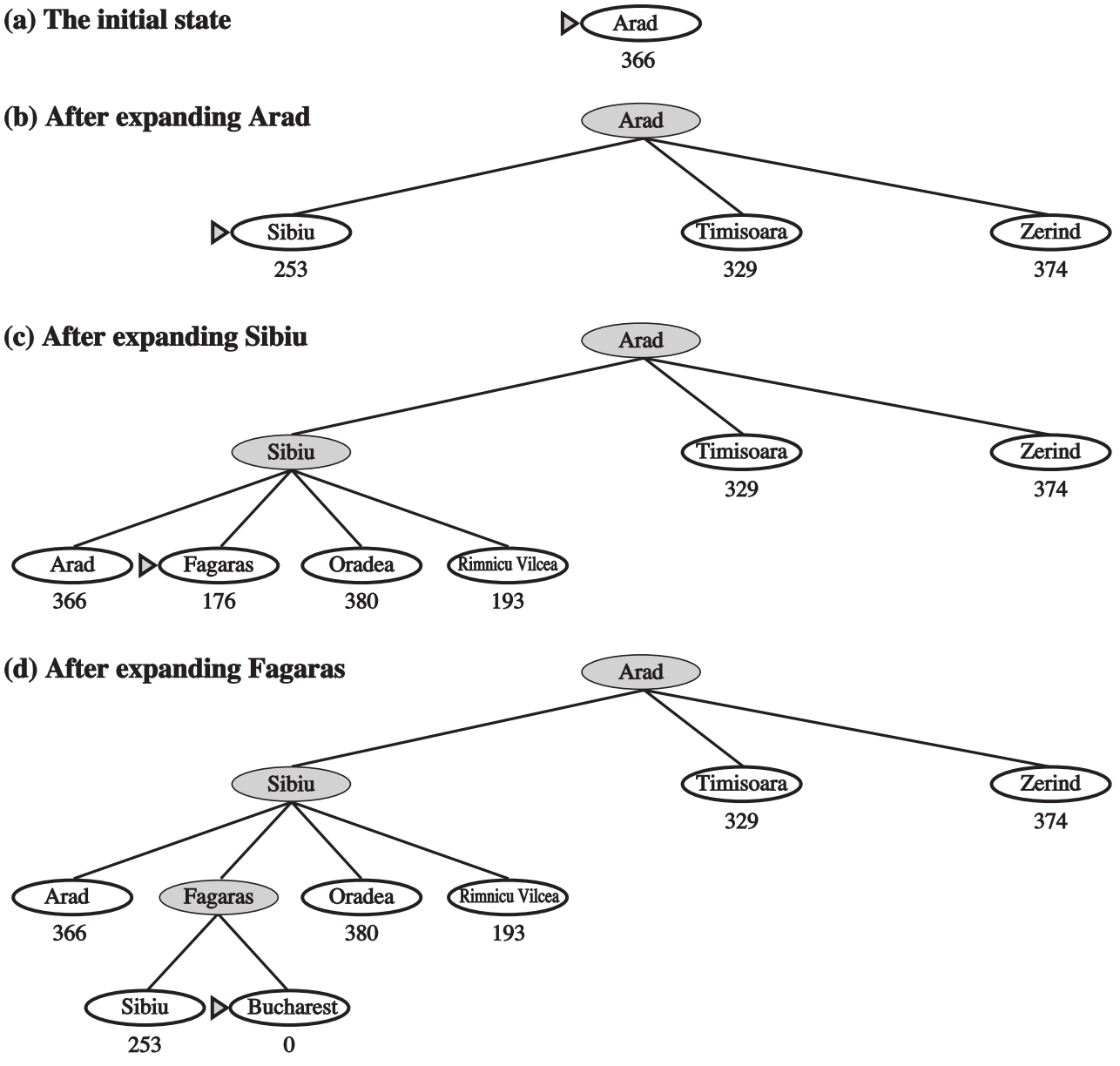


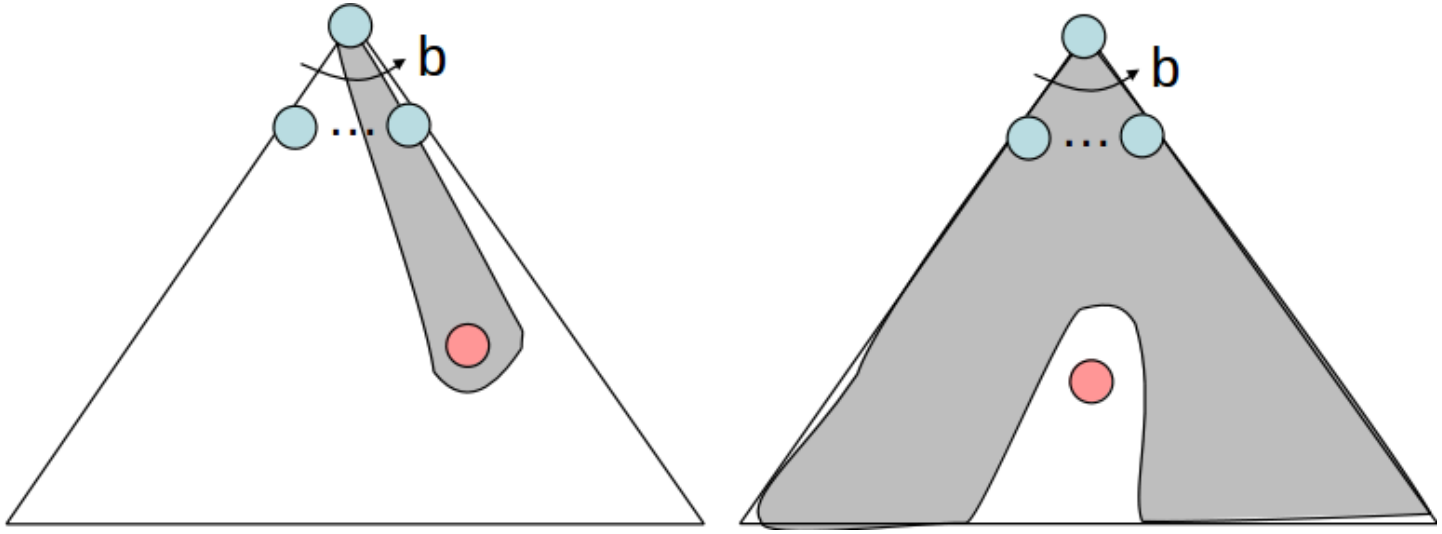
Búsqueda voraz

- *Estrategia*: expandir el nodo n de la frontera para el cual $h(n)$ es menor.
- *Implementación*: la frontera es una

  cola de prioridad

  , usando $h(n)$ como prioridad.


(a) The initial state
Arad
366
(b) After expanding Arad
Arad
Sibiu
Timisoara
Zerind
253
329
374
(c) After expanding Sibiu
Arad
Sibiu
Timisoara
Zerind
329
374
Arad
Fagaras
Oradea
Rimnicu Vilcea
366
176
380
193
(d) After expanding Fagaras
Arad
Sibiu
Timisoara
Zerind
329
374
Arad
Fagaras
Oradea
Rimnicu Vilcea
366
380
193
Sibiu
Bucharest
253
0



En el mejor caso, la búsqueda voraz te lleva directo al objetivo.

En el peor caso, es como un BFS mal guiado.

Propiedades de la búsqueda voraz

- *Completitud*:
  - No, a menos que evitemos ciclos (más sobre esto adelante).
- *Optimalidad*:
  - No, ej. el camino vía Sibiu y Fagaras es 32km más largo que el camino por Rimnicu Vilcea y Pitesti.
- *Complejidad temporal*:
  - $O(b^m)$, a menos que tengamos una buena función heurística.
- *Complejidad espacial*:
  - $O(b^m)$, a menos que tengamos una buena función heurística.

A*



El robot Shakey

- A* se propuso por primera vez en **1968** para mejorar la planificación de robots.
- El objetivo era navegar por una habitación con obstáculos.

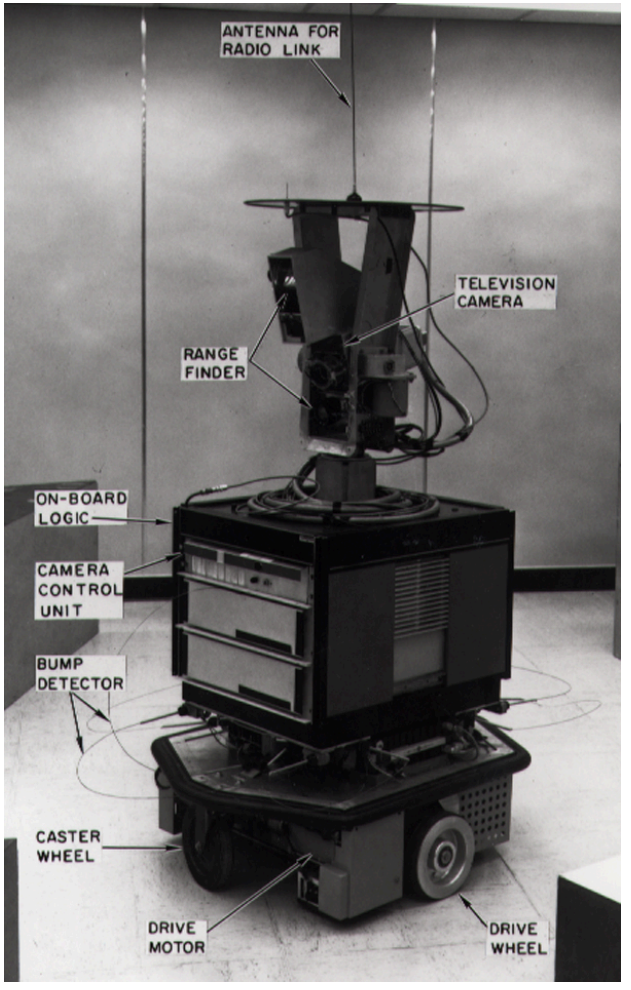


Foto: SRI International — proyecto Shakey (1966–1972).

Shakey el robot (SRI International, 1969)

Ver video →

A*

- El costo uniforme ordena por costo del camino, o

  costo hacia atrás

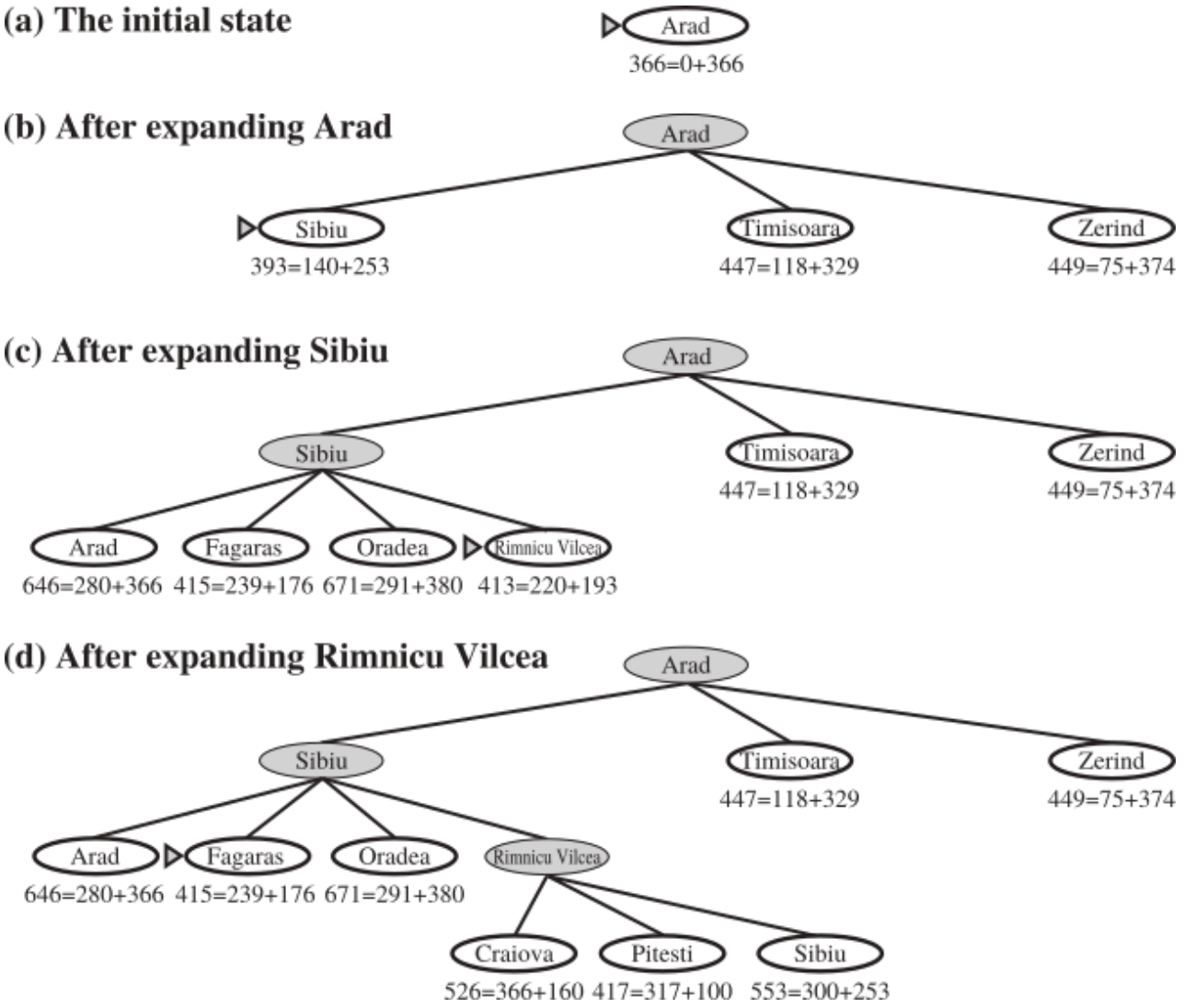
  $g(n)$
- La búsqueda voraz ordena por cercanía al objetivo, o

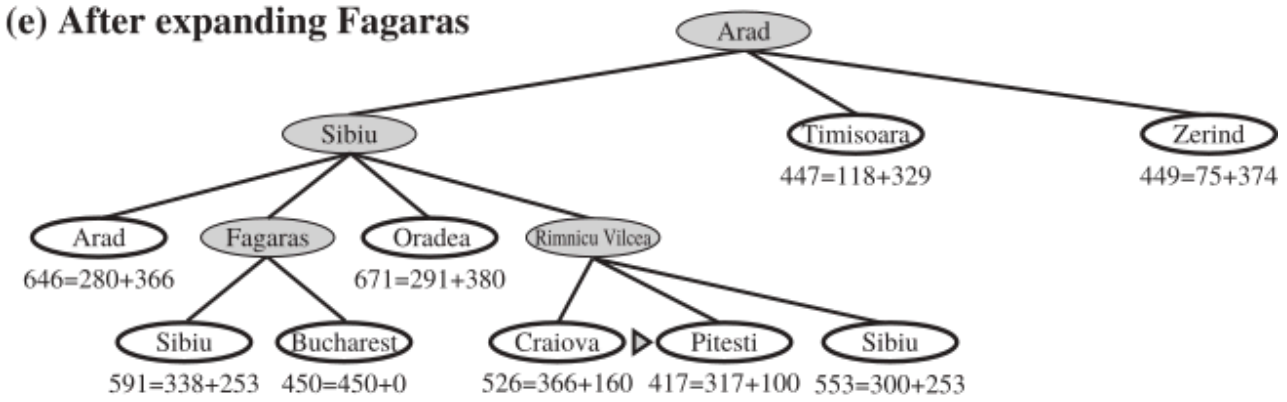
  costo hacia adelante

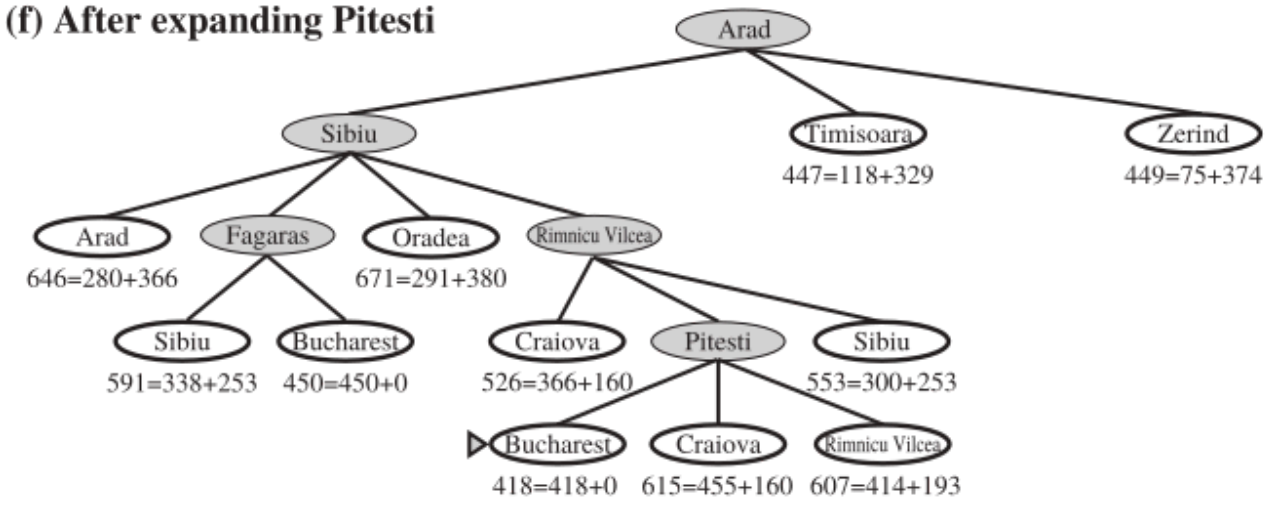
  $h(n)$
- **A*** combina los dos algoritmos y ordena por la suma

$$f(n) = g(n) + h(n)$$

- $f(n)$ es el costo estimado de la solución más barata que pasa por n.


(a) The initial state
Arad
366=0+366
(b) After expanding Arad
Arad
Sibiu
Timisoara
Zerind
393=140+253
447=118+329
449=75+374
(c) After expanding Sibiu
Arad
Sibiu
Timisoara
Zerind
447=118+329
449=75+374
Arad
Fagaras
Oradea
Rimnicu Vilcea
646=280+366
415=239+176
671=291+380
413=220+193
(d) After expanding Rimnicu Vilcea
Arad
Sibiu
Timisoara
Zerind
447=118+329
449=75+374
Arad
Fagaras
Oradea
Rimnicu Vilcea
646=280+366
415=239+176
671=291+380
Craiova
Pitesti
Sibiu
526=366+160
417=317+100
553=300+253





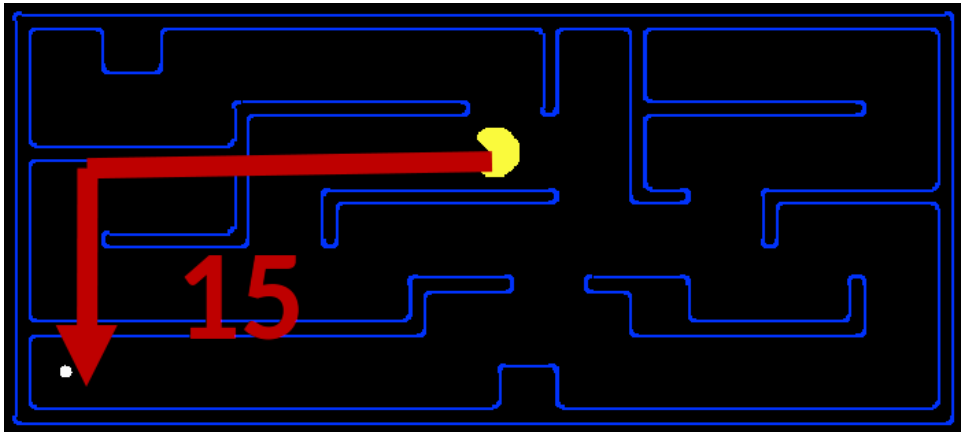
¿Por qué A* no se detiene en el paso (e), si Bucarest ya está en la frontera?

Heurísticas admisibles

Una heurística h es **admisible** si

$$0 \leq h(n) \leq h^*(n)$$

donde $h^*(n)$ es el costo real hasta el objetivo más cercano.



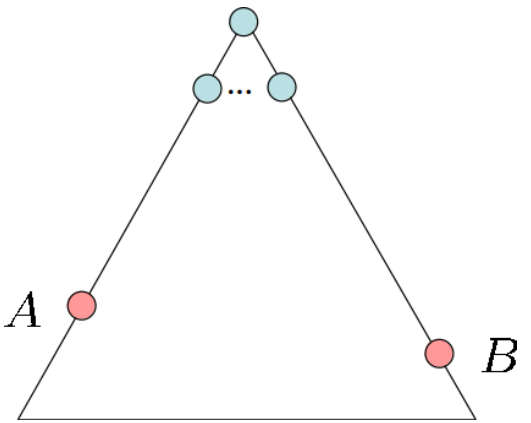
La distancia Manhattan es admisible

Optimalidad de A*

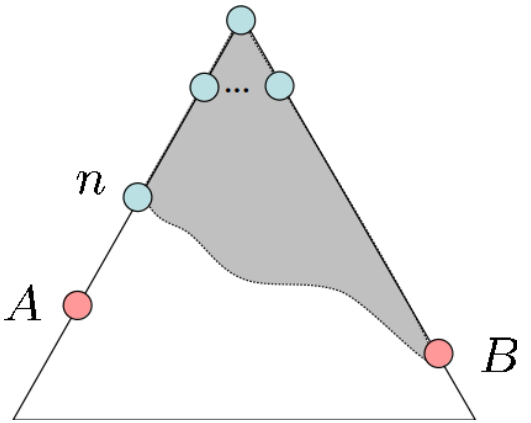
Supuestos:

- A es un nodo objetivo óptimo
- B es un nodo objetivo subóptimo
- h es admisible

Afirmación: A saldrá de la frontera antes que B.



Prueba



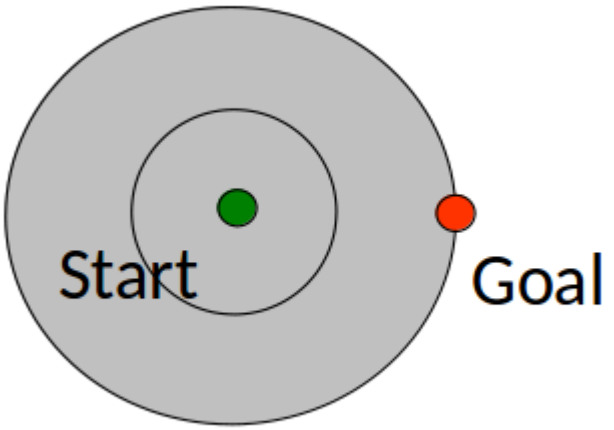
Supongamos que B está en la frontera. Algún ancestro n de A también está en la frontera.

- $f(n) \leq f(A)$
  - $f(n) = g(n) + h(n)$ (por definición)
  - $f(n) \leq g(A)$ (admisibilidad de h)
  - $f(A) = g(A) + h(A) = g(A)$ ($h = 0$ en un objetivo)
- $f(A) < f(B)$
  - $g(A) < g(B)$ (B es subóptimo)
  - $f(A) < f(B)$ ($h = 0$ en un objetivo)
- Por lo tanto, n se expande antes que B.
  - ya que $f(n) \leq f(A) < f(B)$

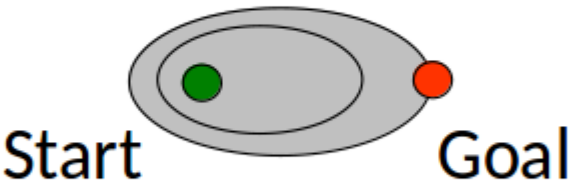
De forma similar, todos los ancestros de A se expanden antes que B, incluido A. Por lo tanto, **A* es óptimo**.

Contornos de A*

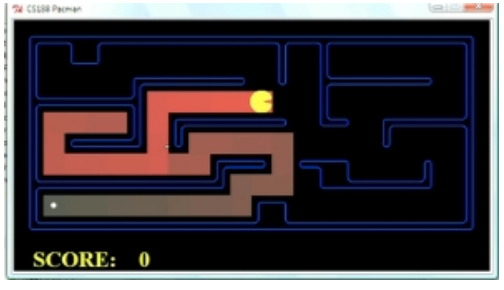
- Supongamos que los costos f son no decrecientes a lo largo de cualquier camino.
- Podemos definir **niveles de contorno** t en el espacio de estados, que incluyen todos los nodos n para los cuales $f(n) \leq t$.



Para UCS ($h(n) = 0$ para todo n), las bandas son circulares alrededor del inicio.



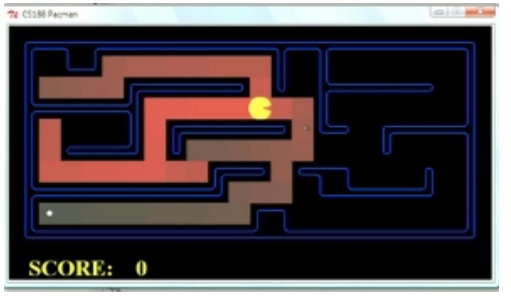
Para A* con heurísticas precisas, las bandas se estiran hacia el objetivo.



Búsqueda voraz



UCS



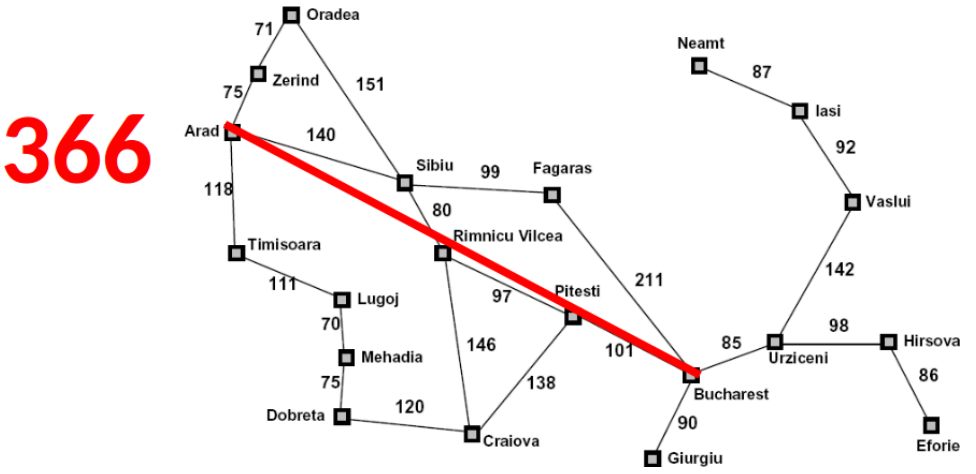
A*

(demo)

Creando heurísticas admisibles

Gran parte del trabajo para resolver problemas de búsqueda difíciles de forma óptima está en encontrar heurísticas admisibles.

Las heurísticas admisibles se pueden derivar de las soluciones exactas de *problemas relajados*, donde hay nuevas acciones disponibles.



Dominancia

- Si h_1 y h_2 son ambas admisibles y $h_2(n) \geq h_1(n)$ para todo n, entonces h_2 **domina** a h_1 y es *mejor* para la búsqueda.
- Dadas heurísticas admisibles cualesquiera h_a y h_b,

$$h(n) = \max(h_a(n), h_b(n))$$

también es admisible y domina a h_a y h_b.

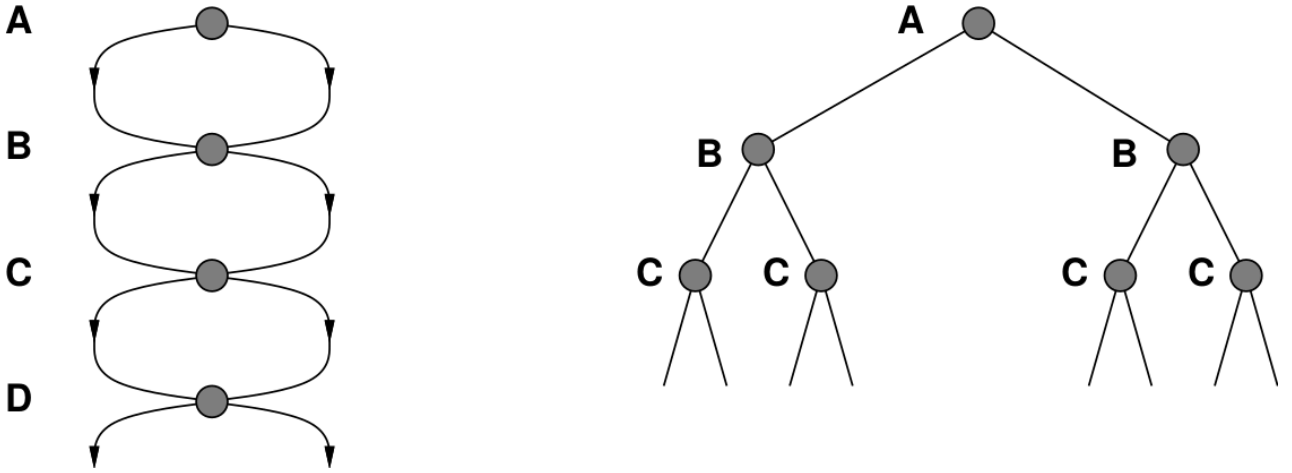
Aprendiendo heurísticas a partir de la experiencia

- Asumiendo un entorno *episódico*, un agente puede **aprender** buenas heurísticas jugando el juego muchas veces.
- Cada solución óptima s^* provee

  ejemplos de entrenamiento

  a partir de los cuales se puede aprender $h(n)$.
- Cada ejemplo consiste de un estado n del camino solución y el costo real $g(s^*)$ de la solución desde ese punto.
- El mapeo $n \rightarrow g(s^*)$ se puede aprender con algoritmos de **aprendizaje supervisado**.
  - Modelos lineales, redes neuronales, etc.

Búsqueda en grafo



La falla al no detectar **estados repetidos** puede convertir un problema lineal en uno exponencial. También puede provocar búsquedas que no terminan.

Los caminos redundantes y los ciclos se pueden evitar

llevando un registro

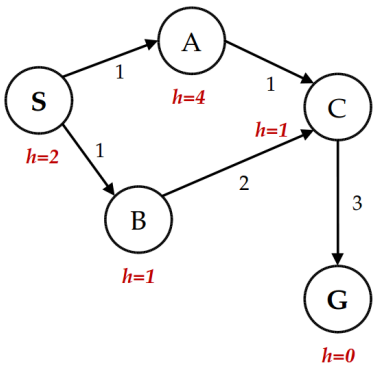
de los estados que ya se *exploraron*. Esto equivale a construir un árbol directamente sobre el grafo del espacio de estados.

```
function GRAPH-SEARCH( problem, fringe) returns a solution, or failure
    closed ← an empty set
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST(problem, STATE[node]) then return node
        if STATE[node] is not in closed then
            add STATE[node] to closed
            fringe ← INSERTALL(EXPAND(node, problem), fringe)
    end
```

.grid[.kol-1-2[

¿A* en grafo sale mal?

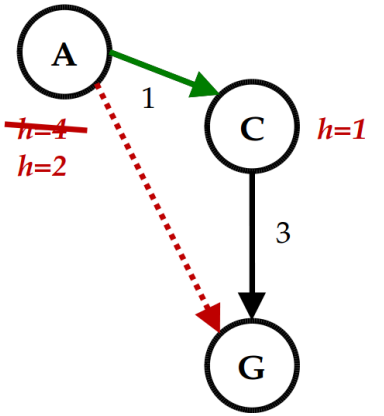
- Empezamos en S y G es un estado objetivo.
- ¿Qué camino encuentra la búsqueda en grafo?



Heurísticas consistentes

Una heurística h es consistente si para todo n y todo sucesor n' generado por cualquier acción a,

$$h(n) \leq c(n, a, n') + h(n').$$



Consecuencias de las heurísticas consistentes:

- $f(n)$ es no decreciente a lo largo de cualquier camino.
- $h(n)$ es admisible.
- Con una heurística consistente, A* con búsqueda en grafo es óptimo.

Ejemplo de repaso: Super Mario



- ¿*Entorno de tarea*?
  - ¿medida de desempeño, entorno, actuadores, sensores?
- ¿*Tipo* de entorno?
- ¿*Problema de búsqueda*?
  - ¿estado inicial, acciones, modelo de transición, prueba de objetivo, costo del camino?
- ¿*Buena heurística*?

Captura de *Super Mario Bros.*, © Nintendo – uso educativo en clase.

A* en acción (recorrido de Mario)



Ver video →

Resumen

- La formulación de un problema normalmente requiere abstraer detalles del mundo real para definir un espacio de estados que se pueda explorar de forma factible.
- Variedad de estrategias de búsqueda no informada (*DFS*, *BFS*, *UCS*, *profundización iterativa*).
- Las funciones heurísticas estiman costos de los caminos más cortos. Una buena heurística puede reducir drásticamente el costo de la búsqueda.
- La búsqueda voraz expande el h más bajo, lo cual resulta ser incompleta y no siempre óptima.
- La búsqueda **A*** expande el $f = g + h$ más bajo. Esta estrategia es completa y óptima.
- La búsqueda en grafo puede ser exponencialmente más eficiente que la búsqueda en árbol.

EAFIT

UNIVERSIDAD

Fin de la Clase 2

Próxima clase: CSP